

Indian Institute of Information Technology Guwahati
Department of Computer Science & Engineering



CS236 – Artificial Intelligence Lab

PROJECT - I
Lunar Lander Agent using PPO (Reinforcement Learning)

Submitted By (Group 2212)

Pradyumn Ojha	Roll No: 2401145
Sagar Raghav	Roll No: 2401169
Shivank Kumar	Roll No: 2401189
Tanishk Gupta	Roll No: 2401213
Vishal Pandey	Roll No: 2401235

Course Instructor (CS235 – Artificial Intelligence): Dr. Kaustuv Nag
Lab Instructor (CS236 – AI Lab): Dr. Upasana Talukdar
Course: CS236 – Artificial Intelligence Lab
Submission Date: 6 April 2026

1. Introduction

The Lunar Lander problem is a classical reinforcement learning task where an agent must learn to land a spacecraft safely on a designated landing pad. The environment provides continuous state observations and discrete actions, making it a standard benchmark for control problems.

In this project, we implement a **Proximal Policy Optimization (PPO)** based agent. Unlike evolutionary approaches, PPO uses policy gradient methods to directly optimize the agent’s behavior through interaction with the environment.

2. Problem Statement

The objective is to train an intelligent agent capable of maximizing the average reward in the Lunar Lander environment.

The constraints are:

- Compatibility with `evaluate_agent.py`
- Execution within time limits
- Efficient policy representation using `.npy` file

3. Environment Description

The Lunar Lander environment consists of:

- **State Space:** 8-dimensional continuous vector
- **Action Space:** 4 discrete actions
 - 0: Do nothing
 - 1: Fire left engine
 - 2: Fire main engine
 - 3: Fire right engine

The reward function encourages safe landing and penalizes crashes and excessive fuel usage.

4. Methodology

4.1 Proximal Policy Optimization (PPO)

PPO is a policy gradient method that optimizes the clipped surrogate objective:

$$L^{CLIP} = E [\min (r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

This ensures stable updates by preventing large policy changes.

4.2 Actor-Critic Architecture

The model consists of two neural networks:

- **Actor Network:** Outputs action probabilities
- **Critic Network:** Estimates state value

Architecture used:

- Input layer: 8 neurons
- Hidden layers: 256 neurons (2 layers)
- Activation: GELU
- Normalization: LayerNorm
- Output layer: 4 actions

Total parameters $\approx 70,000$.

4.3 Advantage Estimation

Generalized Advantage Estimation (GAE) is used:

$$A_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

This improves learning stability.

5. Implementation Details

5.1 Policy Execution

The trained neural network is converted into a flat parameter array and used for inference.

```
h = observation @ W0.T + b0
h = layer_norm + GELU
h = h @ W1.T + b1
h = layer_norm + GELU
logits = h @ Wa.T + ba
action = argmax(logits)
```

5.2 Training Configuration

- Hidden size: 256
- Learning rate(α): $3 \times 10^{-4} \rightarrow 1 \times 10^{-5}$
- Discount factor(γ): 0.99
- GAE parameter(λ): 0.95
- Clip parameter(ϵ): 0.15
- Batch size: 256
- Steps per rollout: 8192
- Epochs per update: 10
- Maximum timesteps: 5 million

Entropy coefficient is annealed from 0.01 to 0.001 for better exploration-exploitation balance.

6. Evaluation Method

The trained agent is evaluated using the provided evaluation script:

- 100 episodes
- First 5 episodes rendered
- Average reward computed

7. Results

Training logs show clear improvement:

- Initial performance: Negative rewards
- Mid training: Gradual improvement
- Final performance: The agent achieved an average reward exceeding 285, with a peak performance of 290.39, indicating near-optimal landing behavior.

The agent learns:

- Stable descent control
- Efficient engine usage
- Safe landing strategy

8. Discussion

Advantages:

- Stable learning due to PPO clipping
- High performance
- Efficient exploration via entropy regularization

Limitations:

- Requires long training time
- Sensitive to hyperparameters

9. Conclusion

The PPO-based agent successfully learns to control the Lunar Lander and achieves high reward performance. This demonstrates the effectiveness of policy gradient methods in solving continuous control problems.

10. References

- OpenAI Gym Documentation
- PPO Research Paper (Schulman et al.)
- CS236 AI Lab Material